Instituto Tecnologico de Costa Rica
Escuela de Ingenieria en Computacion

Reporte de Analisis Estatico con SonarQube

Proyecto analizado: ImageMagick

Asignatura:	IC8071 – Seguridad del Software
Profesor:	Herson Esquivel Vargas
Herramienta:	SonarQube Community Build v25.8.0.112029
Plugin C/C++:	sonar-cxx v2.1.1.2864
Plataforma:	Docker (localhost:9000)
Fecha del analisis:	19 de marzo de 2026
Fecha del reporte:	20 de marzo de 2026

Contents

1 Introduccion **2**

2 Software analizado **2**

3 Configuracion del entorno **2**
3.1 Paso 1: Iniciar SonarQube con Docker 2
3.2 Paso 2: Instalacion del plugin sonar-cxx 2
3.3 Paso 3: Creacion del proyecto 3
3.4 Paso 4: Configuracion del scanner 4
3.5 Paso 5: Ejecucion del analisis 4

4 Resultados **4**
4.1 Historial de analisis y Quality Gate 4
4.2 Metricas generales 5
4.3 Distribucion por modulo 5
4.4 Issues por categoria 5

5 Observaciones y limitaciones **5**

6 Conclusiones **6**

Anexo: Capturas de pantalla **6**

1 Introduccion

Este reporte documenta la configuracion y ejecucion de SonarQube Community mediante Docker para el analisis estatico de seguridad (SAST) del codigo fuente de **ImageMagick**, una biblioteca de codigo abierto para crear, editar y convertir imagenes en mas de 200 formatos.

El trabajo forma parte del Proyecto 1 del curso IC8071 del ITCR, cuyo objetivo es identificar debilidades de software usando herramientas automatizadas bajo el marco de referencia Mitre CWE. Dado que SonarQube Community no incluye soporte nativo para C/C++, fue necesario instalar el plugin **sonar-cxx** antes de ejecutar el analisis.

2 Software analizado

Campo	Valor
Nombre	ImageMagick
Version	1.0 (rama `main`)
Lenguaje	C / C++
Lineas de codigo	273,155
Modulos	`coders`, `filters`, `Magick++`, `MagickCore`, `MagickWand`
Fuente	`apt-get source imagemagick`

3 Configuracion del entorno

3.1 Paso 1: Iniciar SonarQube con Docker

Se levanto el contenedor usando la imagen oficial de la edicion Community:

```
docker run -d \
  --name sonarqube \
  -p 9000:9000 \
  -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
  sonarqube:community
```

La interfaz quedo disponible en `http://localhost:9000`. La instancia usa una base de datos embebida, adecuada para evaluacion pero no recomendada para produccion (SonarQube lo indica con una advertencia en la interfaz).

3.2 Paso 2: Instalacion del plugin sonar-cxx

SonarQube Community Edition no tiene soporte nativo para C/C++; ese soporte esta reservado para las ediciones Developer y Enterprise. El plugin de codigo abierto **sonar-cxx** cubre esta necesidad y fue necesario instalarlo antes de analizar ImageMagick.

Descarga

Se descargo el archivo `.jar` desde el repositorio oficial en GitHub:

```
wget https://github.com/SonarOpenCommunity/sonar-cxx/releases/\
download/cxx-2.1.1/sonar-cxx-plugin-2.1.1.2864.jar
```

La version 2.1.x es compatible con SonarQube 9.x y posteriores, incluyendo la Community Build v25.8.0.112029. La tabla de compatibilidad esta en el repositorio del plugin en GitHub.

Instalacion

El `.jar` se copio al directorio de extensiones del contenedor y se reinicio:

```
docker cp sonar-cxx-plugin-2.1.1.2864.jar \
  sonarqube:/opt/sonarqube/extensions/plugins/

docker restart sonarqube
```

Como alternativa, se puede montar el directorio como volumen al crear el contenedor para evitar repetir la copia si el contenedor se recrea:

```
docker run -d --name sonarqube -p 9000:9000 \
  -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
  -v /ruta/local/plugins:/opt/sonarqube/extensions/plugins \
  sonarqube:community
```

Verificacion

La carga exitosa del plugin se confirma en los logs del contenedor:

```
docker logs sonarqube | grep -i "cxx"
# [INFO] Deploy plugin C++ (Community) / 2.1.1.2864
```

Tambien aparece en **Administration** → **Marketplace** → **Installed** como *C++ (Community)*.

Reglas que aporta sonar-cxx

El plugin registra el perfil de calidad *Sonar way for C++* con reglas en las siguientes categorias:

Categoria	Ejemplos
Manejo de memoria	Desbordamiento de buffer, punteros nulos, use-after-free
Funciones inseguras	`strcpy`, `sprintf`, `gets` sin limite
Inicializacion	Variables usadas sin inicializar
Aritmetica	Desbordamiento de enteros, conversiones inseguras
Integracion externa	Resultados de Cppcheck, Valgrind, Vera++, RATS

3.3 Paso 3: Creacion del proyecto

Desde la interfaz web se creo el proyecto con nombre **ImageMagick**, se genero un token de autenticacion, se fijo la rama principal como `main` y se selecciono el perfil de calidad *Sonar way for C++* habilitado por sonar-cxx.

3.4 Paso 4: Configuracion del scanner

En el directorio raiz del codigo fuente se creo el archivo `sonar-project.properties`:

```
sonar.projectKey=imagemagick
sonar.projectName=ImageMagick
sonar.projectVersion=1.0
sonar.sources=.
sonar.host.url=http://localhost:9000
sonar.token=<TOKEN>

# Propiedades para sonar-cxx (C/C++)
sonar.language=c++
sonar.cxx.file.suffixes=.c,.cpp,.cxx,.cc,.h,.hpp,.hxx
sonar.cxx.errorRecoveryEnabled=true
```

La propiedad `errorRecoveryEnabled` permite que el analizador continue procesando aunque encuentre errores de parseo en archivos con sintaxis no estandar, util en proyectos grandes como ImageMagick.

3.5 Paso 5: Ejecucion del analisis

```
sonar-scanner \
  -Dsonar.projectKey=imagemagick \
  -Dsonar.sources=. \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=<TOKEN> \
  -Dsonar.language=c++ \
  -Dsonar.cxx.file.suffixes=.c,.cpp,.cxx,.cc,.h,.hpp,.hxx
```

El analisis tardo aproximadamente 22 minutos. Durante esa sesion se realizaron cinco ejecuciones incrementales entre las 5:58 PM y las 6:20 PM del 19 de marzo de 2026.

4 Resultados

4.1 Historial de analisis y Quality Gate

El proyecto supero el Quality Gate en las cinco ejecuciones:

Hora	Resultado	Notas
5:58 PM (primer analisis)	Passed	0 issues, 0.0% Cov., 0.0% Dup.
6:03 PM	Passed	+0 issues
6:07 PM	Passed	+0 issues
6:13 PM	Passed	+0 issues
6:20 PM (v1.0)	Passed	+0 issues, +5.0% Dup. (advertencia)

El incremento de duplicaciones en el ultimo analisis genero una advertencia visible en la interfaz, pero no cambio el resultado del gate.

4.2 Metricas generales

Metrica	Nuevo codigo	Total
Issues	0	0
Security Hotspots	0	0
Cobertura	No computada	No computada
Duplicaciones	No computada	5.0%
Lineas de codigo	273,155	

La cobertura no fue calculada porque no se ejecutaron pruebas instrumentadas. Para obtenerla se requeriria integrar `gcov` y pasar los reportes al scanner.

4.3 Distribucion por modulo

Modulo	LoC	Seg.	Conf.	Mant.	Dup.
coders	92,164	0	0	0	5.0%
filters	154	0	0	0	0.0%
Magick++	21,617	0	0	0	0.1%
MagickCore	124,854	0	0	0	3.5%
MagickWand	34,366	0	0	0	13.8%
Total	**273,155**	0	0	0	5.0%

4.4 Issues por categoria

La seccion Issues confirmo cero problemas en todas las categorias ("No Issues. Hooray!"):

Categoria	Cantidad
Security / Reliability / Maintainability	0
Blocker / High / Medium / Low / Info	0
Consistency / Intentionality / Adaptability	0
Total	**0**

5 Observaciones y limitaciones

- SonarQube Community, incluso con el plugin sonar-cxx, no realiza *taint analysis* profundo. Ese nivel de analisis esta disponible en las ediciones Developer y Enterprise. El plugin complementa con reglas especificas de C/C++ pero no es equivalente.
- Cero issues no descarta debilidades. SonarQube es susceptible a falsos negativos en codigo C con manejo manual de memoria (desbordamientos de buffer, use-after-free). Es necesario complementar con Semgrep y LLMs para una cobertura mas amplia frente a CWEs relevantes en C/C++.
- El analisis cubre unicamente el codigo fuente de ImageMagick, sin dependencias externas.

6 Conclusiones

El analisis de 273,155 lineas de codigo de ImageMagick con SonarQube Community y el plugin sonar-cxx arrojo **0 issues** y **0 Security Hotspots**, con el Quality Gate aprobado en los cinco analisis realizados. La unica observacion fue el incremento de duplicaciones en el analisis final (+5%), que genero una advertencia pero no afecto el gate.

Los resultados deben interpretarse con cautela: la ausencia de issues no implica que el codigo este libre de debilidades de seguridad. Este analisis es un primer paso que debe complementarse con Semgrep y revision mediante LLMs para cobertura mas amplia.

Anexo: Capturas de pantalla

- **Capturas 1 y 2:** Vista general del proyecto con Quality Gate aprobado, 0 nuevos issues y 0 Security Hotspots.
- **Captura 3:** Historial de actividad con los cinco analisis del 19 de marzo (5:58 – 6:20 PM).
- **Captura 4:** Seccion Issues mostrando "No Issues. Hooray!" en todas las categorias.
- **Captura 5:** Vista Code con la distribucion de LoC y metricas por modulo.